MemAgent: Reshaping Long-Context LLM with Multi-Conv RL-based Memory Agent

ICLR 2026 (Oral)

Hongli Yu et al.
Presented by: Fengnan Li

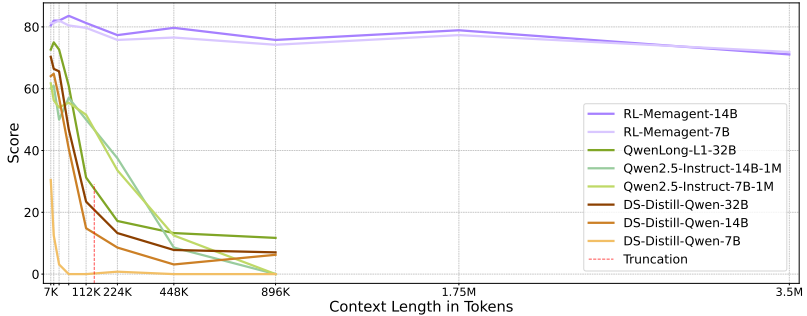
Duke B&B

Oct, 2026

Outline

1. Background & Motivation
2. The MemAgent Method
3. Experimental Setup
4. Results
5. Discussion & Conclusion

Long-Context LLMs Fail Well Before Their Nominal Window



RULER-HQA accuracy vs. input length. Red dashed line: beyond it, the 128K-window baselines see truncated input.

- Baselines decay long before their window is full; the 1M-token Qwen2.5 models score 0 at 896K
- RL-MemAgent-14B uses an 8K window, yet goes from 80.5 at 7K to 71.1 at 3.5M tokens

What a long-context solution should deliver

(1) unbounded input length (2) no performance cliff as length grows (3) cost linear in length

Three Existing Routes to Long Context

1. Length extrapolation + continued pretraining

Rescale RoPE positions (NTK, PI, YaRN, DCA), then train on longer text. Attention stays $O(n^2)$: slow on extreme lengths, and accuracy still degrades.

2. Efficient architectures

Sparse attention (Longformer, StreamingLLM), linear attention, RNNs/SSMs (Mamba, RWKV). Usually need training from scratch; sparse patterns are hand-designed.

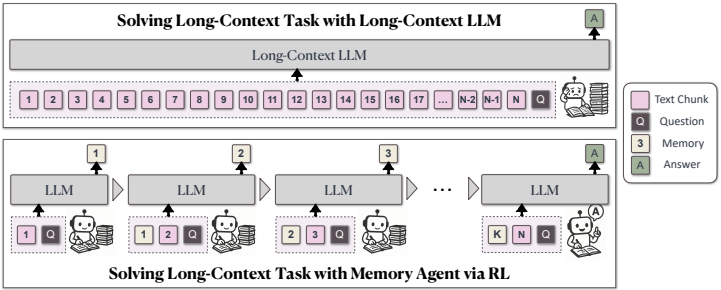
3. Compression and memory

Token compression (LLMLingua), memory modules (Titans, RMT), retrieval memory agents (Mem0). Struggle to extrapolate; extra modules change the standard generation process.

MemAgent's route

Keep the base LLM unchanged. Use RL to teach it to read in chunks and keep a fixed-size memory written in plain text.

Core Idea: Read Like a Human, Keep Notes



- Split the document into chunks; at each step the LLM sees only question + current chunk + memory
- Memory: $\leq$1,024 ordinary tokens written by the LLM, overwritten (not appended) after every chunk; the last step answers from question + memory alone
- No change to architecture, positional encoding, or attention: any instruct LLM can be the agent

RL decides what to keep

Reward comes only from a correct final answer: the memory learns to keep answer-critical facts and drop distractors.

Workflow: Two Prompts, One 8K Window

Context processing (once per chunk)

You are presented with a problem, a section of an article that may contain the answer, and a previous memory. Please read the section carefully and *update the memory* with new information that helps to answer the problem, while retaining all relevant details from the previous memory.

```
<problem>{prompt}</problem>
<memory>{memory}</memory>
<section>{chunk}</section>
```

Updated memory:

Answer generation (once, at the end)

You are presented with a problem and a previous memory. Please answer the problem based on the previous memory and put the answer in `\boxed{}`.

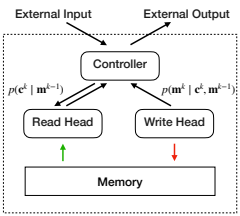
```
<problem>{prompt}</problem>
<memory>{memory}</memory>
```

Your answer:

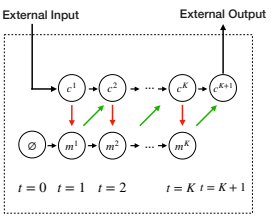
8K window	query	chunk	memory	output	rest
tokens	1,024	5,000	1,024	1,024	chat template

- A document of N tokens takes $\lceil N/5000 \rceil$ memory updates + 1 answer call, each of constant size $\Rightarrow$ total cost $O(N)$
- 512K tokens $\approx$ 100 updates; 3.5M tokens = 700 updates

A Formal View: Memory as a Latent Variable



Architecture of MemAgent



Graphical Model of MemAgent

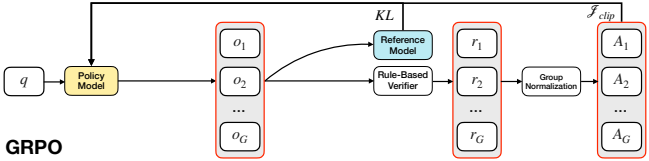
- Standard LM: $p(\mathbf{x}_{1:N}) = \prod_n p(x_n \mid \mathbf{x}_{1:n-1})$ keeps the whole history in context. MemAgent streams K chunks $\mathbf{c}^k$ through memories $\mathbf{m}^k$ of fixed length M ($\mathbf{m}^0 = \varnothing$):

$$p(\mathbf{x}_{1:N}) = \sum_{\mathbf{m}^{1:K-1}} \prod_{k=1}^{K} \underbrace{p(\mathbf{c}^k \mid \mathbf{m}^{k-1})}_{\text{read}} \underbrace{p(\mathbf{m}^k \mid \mathbf{c}^k, \mathbf{m}^{k-1})}_{\text{write}}$$

Each step sees only $(\mathbf{c}^k, \mathbf{m}^{k-1})$: $O(C+M)$ per step, $O(N)$ overall

Intuition: an RNN whose hidden state is plain text; the writes form an MDP, and RL learns which memory states lead to correct answers.

Background: RL with Verifiable Rewards (GRPO → DAPO)



GRPO rollout and update

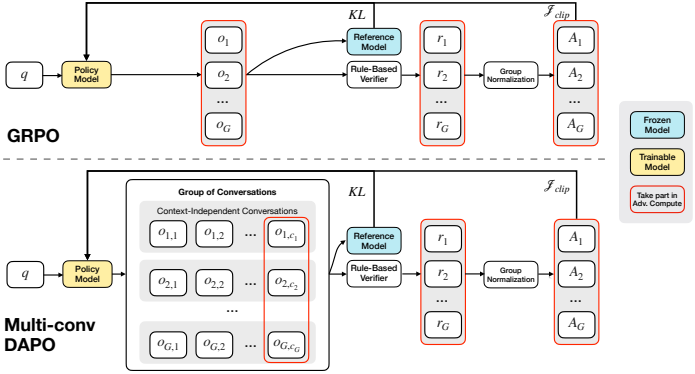
- GRPO (Shao et al., 2024): sample G responses $o_1, \ldots, o_G$ for prompt q; a rule-based verifier gives $r_i \in \{0, 1\}$; PPO-style clipped update with no value network, using

$$\hat{A}_{i,t} = \frac{r_i - \text{mean}(\{r_j\}_{j=1}^{G})}{\text{std}(\{r_j\}_{j=1}^{G})} \quad \text{for every token } t \text{ of } o_i$$

- DAPO (Yu et al., 2025): decoupled clip range $[1 - \varepsilon_{\text{low}}, 1 + \varepsilon_{\text{high}}]$, loss averaged over *all tokens* in the group, dynamic sampling, overlong reward shaping
- Dr. GRPO (Liu et al., 2025): drop the std normalization, which biases the update by question difficulty

All of these assume one prompt → one response in one context. A MemAgent rollout is a chain of separate conversations.

Multi-Conv DAPO: RL over Context-Independent Conversations



Red frames: quantities used to compute the advantage.

- Agent RL usually concatenates turns into one growing context (Search-R1, Agent-R1), which would defeat the fixed memory
- Here, rollout i is a chain of n_i separate conversations: $n_i - 1$ memory updates $+ 1$ answer
- Only the final conversation is scored; its advantage is shared by every conversation of that rollout

Multi-Conv DAPO: The Objective

- Outcome reward from a rule-based verifier: $R_i = \mathbf{1}[\texttt{is_equiv}(y, \hat{y}_i)]$
- Advantage, shared by all conversations j and tokens t of rollout i (no std, following Dr. GRPO):

$$\hat{A}_{i,j,t} = R_i - \mathrm{mean}(\{R_i\}_{i=1}^{G})$$

- Loss over the new `(group, conversation, token)` dimension:

$$\mathcal{J}(\theta) = \mathbb{E}\left[\frac{1}{\sum_{i=1}^{G}\sum_{j=1}^{n_i}|o_{i,j}|}\sum_{i=1}^{G}\sum_{j=1}^{n_i}\sum_{t=1}^{|o_{i,j}|}\Big(\mathcal{C}_{i,j,t} - \beta\, D_{\mathrm{KL}}(\pi_\theta \,\|\, \pi_{\mathrm{ref}})\Big)\right]$$

$$\mathcal{C}_{i,j,t} = \min\Big(r_{i,j,t}(\theta)\hat{A}_{i,j,t},\ \mathrm{clip}\big(r_{i,j,t}(\theta), 1-\varepsilon_{\mathrm{low}}, 1+\varepsilon_{\mathrm{high}}\big)\hat{A}_{i,j,t}\Big), \quad r_{i,j,t}(\theta) = \frac{\pi_\theta(o_{i,j,t} \mid q, o_{i,j,<t})}{\pi_{\theta_{\mathrm{old}}}(o_{i,j,t} \mid q, o_{i,j,<t})}$$

- Token-level normalization over *all* conversations: long trajectories contribute more tokens to the gradient

Coarse credit assignment

One binary reward is spread evenly over every memory write in the rollout (about 6–12 per training document): a correct answer reinforces them all, including writes that dropped useful facts. There is no step-level reward.

Training Recipe

- Backbones: Qwen2.5-7B-Instruct and Qwen2.5-14B-Instruct; multi-conversation RL built on verl
- 8K context window throughout training, split as on the workflow slide

Stage	Data
I: basic memory	32,768 synthetic QA, $\sim$28K tokens each: a HotpotQA question with its gold paragraphs hidden among distractor Wikipedia articles (RULER recipe). Trained to convergence ($\sim$400 steps)
II: transfer	2,560 harder instances up to 60K tokens: DocQA-RL-1.6K (from QwenLong-L1) mixed with Stage I data

- Filtering: drop questions Qwen2.5-7B (Base or Instruct) answers *without any context* (best-of-2 = 100%); about 50% of 80,000 HotpotQA questions removed
- Hyperparameters: group size 16, rollout batch 256, learning rate 10^{-6}

Trained on documents of at most 60K tokens; evaluated on up to 3.5M, about 60$\times$ longer.

Benchmarks and Baselines

Benchmark	What it tests	Lengths
RULER-HQA	HotpotQA multi-hop QA with gold paragraphs among distractors; same recipe as Stage I data	7K–3.5M
LongBench-QA	2Wiki, HotpotQA, MuSiQue, NarrativeQA, Qasper: shorter but information-dense, varied genres	native
NIAH (RULER)	one key–value needle; harder haystack (essays) and needle (UUID) at levels 2–3	8K–512K
LongBench-SUM	GovReport and QMSum summarization (ROUGE recall)	native

- Baselines: QwenLong-L1-32B (RL-trained for long documents), Qwen2.5-Instruct-7B/14B-1M, DeepSeek-R1-Distill-Qwen-7B/14B/32B, Qwen2.5-Instruct-7B/14B/32B
- Input budget: 128K-window models get 120,000 input tokens, 1M models get 990,000; longer inputs are truncated; 10,000 output tokens
- Metric: `sub_em`: correct if the answer contains every element of the ground truth (after normalization)
- MemAgent: same 8K window as in training (1,024-token memory, 5,000-token chunks)

RULER-HQA: From 7K to 3.5M Tokens

Model	7K	14K	28K	56K	112K	224K	448K	896K	1.75M	3.5M
QwenLong-L1-32B	72.66	75.00	72.66	60.94	31.25	17.19	13.28	11.72	N/A	N/A
Qwen2.5-Inst-14B-1M	60.16	60.94	50.00	57.03	50.00	37.50	8.59	0.00	N/A	N/A
Qwen2.5-Inst-7B-1M	61.72	56.25	53.91	55.47	51.56	33.59	12.50	0.00	N/A	N/A
DS-Distill-Qwen-32B	70.31	66.41	65.62	46.88	23.44	13.28	7.81	7.03	N/A	N/A
DS-Distill-Qwen-14B	64.06	64.84	57.03	40.62	14.84	8.59	3.12	6.25	N/A	N/A
DS-Distill-Qwen-7B	30.47	12.50	3.12	0.00	0.00	0.78	0.00	0.00	N/A	N/A
Qwen2.5-Inst-32B	69.53	64.84	60.16	51.56	44.53	21.88	14.06	7.03	N/A	N/A
Qwen2.5-Inst-14B	75.00	67.19	57.03	54.69	44.53	21.88	10.94	2.34	N/A	N/A
Qwen2.5-Inst-7B	52.34	57.03	51.56	44.53	32.81	13.28	6.25	1.56	N/A	N/A
RL-MemAgent-14B	80.47	**82.03**	**82.03**	**83.59**	**81.25**	**77.34**	**79.69**	**75.78**	**78.91**	71.09
RL-MemAgent-7B	**81.25**	81.25	**82.03**	80.47	79.69	75.78	76.56	74.22	77.34	**71.88**

Accuracy (%). Best per column in bold; N/A: beyond the model's input budget.

- RL-MemAgent loses <10 points from 7K to 3.5M; the 7B model is as good as the 14B
- QwenLong-L1-32B is strong at short lengths but falls to 31.25 at 112K; the 1M models reach 0 at 896K

Caveat

RULER-HQA is built like the Stage I training data, so it is in-distribution for MemAgent; beyond 120K the 128K baselines see truncated input.

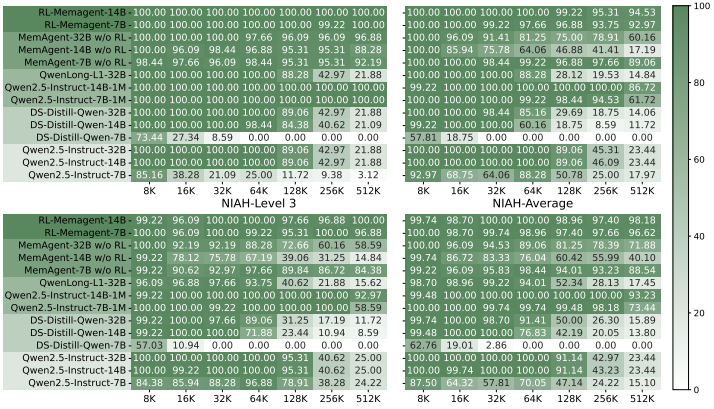
LongBench: Real Documents, QA and Summarization

	LongBench-QA						LongBench-SUM	
Model	2Wiki	HQA	MuSiQue	NQA	Qasper	AVG	GovReport	QMSum
QwenLong-L1-32B	83.0	69.5	51.0	**26.0**	24.0	50.7	16.29	23.74
Qwen2.5-Inst-14B-1M	70.5	65.5	35.0	22.0	22.0	43.0	19.34	29.84
Qwen2.5-Inst-7B-1M	67.5	58.5	26.0	21.5	24.0	39.5	19.26	29.24
DS-Distill-Qwen-32B	**83.5**	69.0	47.5	24.0	21.0	49.0	15.99	23.93
DS-Distill-Qwen-14B	**83.5**	67.0	42.0	22.0	22.0	47.3	17.25	25.07
DS-Distill-Qwen-7B	47.0	31.5	7.5	4.0	17.5	21.5	19.10	20.62
Qwen2.5-Inst-32B	68.5	66.0	37.0	24.5	22.5	43.7	14.90	29.08
Qwen2.5-Inst-14B	67.0	63.5	39.0	20.0	20.5	42.0	19.04	29.09
Qwen2.5-Inst-7B	52.5	59.0	24.0	19.0	19.0	34.7	19.26	28.99
RL-MemAgent-14B	79.0	**73.0**	**52.0**	25.0	26.0	**51.0**	**21.80**	**31.39**
RL-MemAgent-7B	74.0	69.5	47.0	21.5	**29.0**	48.2	19.34	31.27

QA: accuracy (%). Summarization: average ROUGE-1/2/L recall. Best per column in bold.

- RL-MemAgent-14B has the best QA average (51.0), narrowly ahead of QwenLong-L1-32B (50.7) at less than half the size
- RL adds +13.5 (7B) and +9.0 (14B) points over the Qwen2.5-Instruct backbones
- Summarization is not retrieval, yet MemAgent is still best on both datasets, by small margins

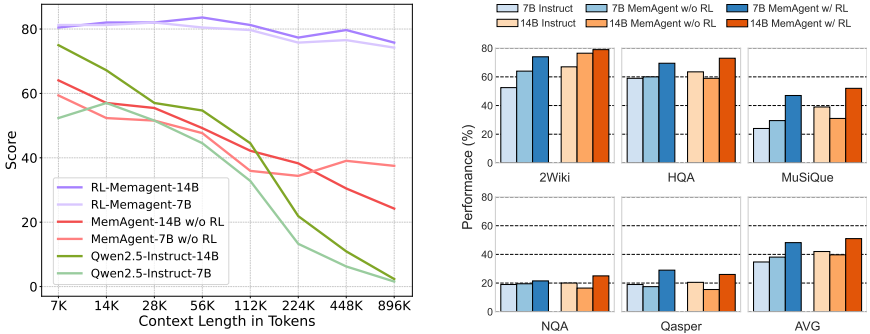
Needle in a Haystack (RULER NIAH, 8K–512K)



Three needle difficulty levels and their average

- RL-MemAgent: average $\geq$96.6% at every length, even at 512K (>100 memory updates)
- Without RL, MemAgent-14B falls to 40.1 at 512K; best baseline (Qwen2.5-Inst-14B-1M): 93.2

Is RL Necessary? The Workflow Alone vs. RL-Trained



Left: RULER-HQA. Right: LongBench-QA. "w/o RL" = the same workflow with the untrained backbone.

- RULER-HQA: the untrained workflow beats the backbone at long lengths but still decays steadily
- LongBench-QA: the untrained workflow gives marginal or even negative gains over the backbone

The chunk-and-memory workflow is not enough by itself: RL training is what makes the memory robust and general.

Comparison with RAG Agents

Method	7K	14K	28K	56K	112K	224K	448K	896K	1.75M	3.5M
RAG + Qwen2.5-14B, K=2	57.03	54.69	51.56	54.69	53.12	50.00	52.34	49.22	48.44	48.44
RAG + Qwen2.5-14B, K=8	78.12	78.91	77.34	81.25	76.56	78.12	77.34	74.22	70.31	64.84
RL-MemAgent-14B	80.47	82.03	82.03	83.59	81.25	77.34	79.69	75.78	78.91	71.09
RAG + Qwen2.5-7B, K=8	75.00	75.00	75.78	74.22	74.22	77.34	72.66	68.75	64.06	64.84
RL-MemAgent-7B	81.25	81.25	82.03	80.47	79.69	75.78	76.56	74.22	77.34	71.88

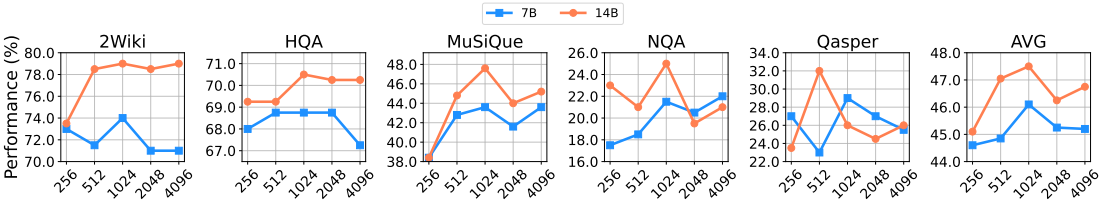
RULER-HQA: retrieve the top-K Wikipedia articles with text-embedding-3-large, then answer with Qwen2.5.

Method	2Wiki	HQA	MuSiQue	NQA	Qasper
best RAG + Qwen2.5-14B (C=2048, K=8)	77.50	68.50	42.00	21.00	27.50
RL-MemAgent-14B	79.0	73.0	52.00	25.00	26.00
best RAG + Qwen2.5-7B (C=1024, K=6)	54.50	57.50	29.50	17.00	25.00
RL-MemAgent-7B	74.00	69.50	47.00	21.50	29.00

LongBench-QA: best RAG setting per backbone (chunk size C, top-K).

- RULER-HQA: RAG with K=8 is within a few points up to 896K; the gap opens at 1.75M–3.5M
- LongBench-QA: MemAgent beats the best RAG setting on 4 of 5 datasets (14B) and on all 5 (7B)

How Large Should the Memory Be?



LongBench-QA vs. memory length 256–4,096 tokens

- LongBench-QA: a 1,024-token memory gives the best average for both 7B and 14B; 256 is clearly too small
- NIAH: every memory size from 256 to 4,096 averages $\geq$92.2% at all lengths up to 512K; one needle needs little room
- Holding memory + chunk fixed at 6,024 tokens: NIAH stays $\geq$94.5%; LongBench-QA is best at memory 1,024 / chunk 5,000

Trade-off

A larger memory stores more but is harder to manage and invites redundancy; a smaller one runs out of room. The default (1,024) is a reasonable sweet spot.

Case Study: What the Memory Learns

Question (2-hop): *The director of the romantic comedy "Big Stone Gap" is based in what New York city?*

Round 1: a distractor

Chunk: an article on *Ghost*, a production team "based in New York City"

Memory: keeps it as a possible lead: "Ghost is a Swedish record producing and songwriting team ... based in New York City."

Round 2: nothing relevant

Chunk: no relevant document

Memory: unchanged, plus a note: "The problem remains unanswered based on the provided section."

Round 3: gold evidence

Chunk: *Big Stone Gap* was "written and directed by Adriana Trigiani"; Trigiani is "based in Greenwich Village, New York City"

Memory: adds both facts and concludes the director is Trigiani, based in Greenwich Village

- Later rounds leave the memory unchanged; final answer: Greenwich Village, New York City (correct)

Learned behaviors, all emerging from RL rather than architecture

Keep query-related leads early · update as soon as matching evidence appears · ignore distractors

Trajectory of RL-MemAgent-14B

Where Does Evidence Get Lost? Position Probe and Failure Modes

Probe: two groups of key facts placed at fixed positions in RULER-HQA (0% = start, 100% = end) vs. random placement

	RL-MemAgent-14B		RL-MemAgent-7B	
Key-fact positions	avg. change	range	avg. change	range
(0%, 20%)	+1.90	[−3.91, +4.69]	+2.35	[−1.56, +5.47]
(0%, 100%)	+2.79	[−3.12, +7.82]	+2.23	[0.00, +4.69]
(20%, 80%)	0.00	[−3.13, +5.47]	+0.56	[−0.78, +3.13]
(40%, 60%)	−0.11	[−3.12, +3.13]	+1.45	[−3.91, +5.47]
(80%, 100%)	+0.67	[−2.35, +3.13]	+1.00	[−0.78, +4.69]

Accuracy change vs. random placement (points); range over 7 lengths, 7K–448K.

(0%, 100%) stresses overwriting (the first fact must survive the whole document), (40%, 60%) stresses lost-in-the-middle: no setting hurts (average change −0.11 to +2.79)

Failure patterns

- Overwriting: memory fills with irrelevant notes; the key fact is appended, truncated, and later lost
- Missed multi-hop evidence: a fact arrives before its bridging entity is known, so it looks irrelevant
- Primacy bias: an early wrong reading ("country" as nation, not countryside) gets reinforced

Strengths and Limitations

Strengths

Simple, architecture-agnostic · linear $O(N)$ cost · trained on $\leq$60K tokens, works at 3.5M

Limitation 1: Query-conditioned memory

$\mathbf{m} = f(\text{document}, \text{question})$, not $f(\text{document}) \Rightarrow$ no memory for questions asked later

Limitation 2: Verifiable rewards only

QA fits $R = \mathbf{1}[\hat{y} = y]$; open-ended tasks $\Rightarrow$ hard credit assignment

Limitation 3: Fixed memory bottleneck

3.5M $\rightarrow$ 1,024 tokens $\Rightarrow$ fails when the needed information grows with document length

Assessment

Is it worth reading? Yes!

- Simple, intuitive idea that is easy to follow

Is it worth implementing? Maybe.

- Open-source implementation on GitHub (1.1K+ stars)
- Expensive to train: $\sim$2.5K–4K H100 GPU hours (7B), $\sim$4K–7K (14B)

Thank You!

Questions?